

Transformer Expressivity

universal approximation, contextual mapping, positional encoding, depth-width 조건

이번 주의 질문은 “Transformer가 실제로 잘 학습되는가?”가 아니라 더 먼저 오는 질문이다. 주어진 **architecture**가 어떤 함수들을 표현할 수 있는가? Yun et al.은 positional encoding이 없는 Transformer가 연속 permutation-equivariant sequence-to-sequence 함수를 보편적으로 근사하고, trainable positional encoding을 넣으면 고정 길이의 임의의 연속 sequence-to-sequence 함수까지 근사할 수 있음을 보였다. 증명의 중심은 self-attention이 입력 전체의 문맥을 구별하는 **contextual mapping**을 만들고, token-wise feed-forward layer가 그 식별자를 원하는 출력으로 변환한다는 구조다. 이어서 attention matrix 자체의 표현력과 head dimension의 low-rank bottleneck을 분석해, “fixed width로 universal”이라는 문장이 실제로 무엇을 뜻하고 무엇을 뜻하지 않는지 분리한다.

Primary readings

Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, Sanjiv Kumar, **Are Transformers universal approximators of sequence-to-sequence functions?**, ICLR 2020.

Srinadh Bhojanapalli, Chulhee Yun, Ankit Singh Rawat, Sashank J. Reddi, Sanjiv Kumar, **Low-Rank Bottleneck in Multi-head Attention Models**, ICML 2020.

Contents

1	먼저 맞춰 둘 기호와 전체 그림	3
1.1	논문마다 다른 행렬 convention	3
1.2	이번 주의 세 층위	3
1.3	이번 주의 결론을 먼저 한 줄로	4
2	Transformer block의 수학과 permutation symmetry	5
2.1	원 논문 형태의 self-attention	5
2.2	token-wise FFN	5
2.3	왜 positional encoding이 없으면 permutation equivariant인가	5
2.4	이 symmetry가 의미하는 제한	6
3	Universal Approximation을 정확히 읽기	7
3.1	고전적 UAT와 이번 정리의 차이	7
3.2	L^p 함수 거리	7
3.3	함수 class 정의	7
3.4	Theorem 2: positional encoding 없이도 fixed-width UAT	8
3.5	왜 이 결과가 처음 보면 이상한가	8
4	증명의 중심: contextual mapping	9
4.1	왜 token-wise FFN만으로는 부족한가	9
4.2	Contextual mapping의 정의	9
4.3	UAT proof의 세 단계	9
4.4	Attention이 contextual ID를 만든다는 의미	10
5	Positional encoding이 symmetry를 어떻게 깨는가	12
5.1	왜 위치 정보가 필요할까	12
5.2	trainable positional encoding	12
5.3	Theorem 3	12
5.4	“positional encoding이 있으면 universal”의 정확한 범위	13
5.5	PE는 단순한 부가 정보가 아니라 symmetry-breaking device	13
6	Attention matrix 자체의 표현력과 low-rank bottleneck	14
6.1	질문을 한 layer 수준으로 내리기	14
6.2	softmax 역문제	14
6.3	Representation Theorem	14
6.4	충분한 차원에서의 explicit construction	15
6.5	왜 $d_k < n$ 이면 bottleneck이 생기는가	15
6.6	multi-head에서 생기는 직관	15
7	Depth와 Width: fixed-width universal의 실제 비용	17
7.1	폭이 작다는 것과 효율적이라는 것은 다르다	17
7.2	왜 curse of dimensionality가 보이는가	17
7.3	Depth의 역할	17
7.4	width condition을 어떻게 읽을 것인가	18

8	UAT가 말하지 않는 것	19
8.1	1. SGD learnability를 보장하지 않는다	19
8.2	2. sample complexity를 보장하지 않는다	19
8.3	3. length generalization을 보장하지 않는다	19
8.4	4. computational complexity를 보장하지 않는다	19
8.5	5. attention weight의 해석 가능성을 보장하지 않는다	19
8.6	Expressivity, computability, learnability, generalization	20
9	발표 준비용 압축 정리	21
9.1	칠판에 반드시 남길 여덟 식	21
9.2	발표 순서 추천	21
9.3	자주 나오는 질문	22
9.4	흔한 수학적 실수 체크리스트	22
9.5	30초 결론	22
	참고문헌 및 원문에서 확인할 위치	24

1

먼저 맞춰 둘 기호와 전체 그림

1.1 논문마다 다른 행렬 convention

Transformer 이론 논문에서 가장 먼저 생기는 혼란은 token을 행으로 둘지 열로 둘지다. 현대 구현 설명에서는 보통 $X \in \mathbb{R}^{n \times d}$ 로 token을 행으로 두지만, Yun et al.과 Bhojanapalli et al.은

$$X \in \mathbb{R}^{d \times n}$$

처럼 token을 열로 둔다. 이 노트는 원 논문의 증명을 따라가기 위해 후자를 기본으로 쓴다.

개념	이 노트 / Yun et al.	흔한 구현 표기
입력 sequence	$X \in \mathbb{R}^{d \times n}$	$X_{\text{row}} \in \mathbb{R}^{n \times d}$
token 위치	column $X_{:,j}$	row $X_{j,:}$
permutation	XP	PX_{row}
query/key	$W_Q X, W_K X$	$X_{\text{row}} W_Q$ 등
projection		
attention score	$(W_K X)^\top (W_Q X)$	$QK^\top$
softmax	column-wise	row-wise가 혼함
sequence length	n	n
embedding	d	d
dimension		
head size	m 또는 d_h	d_h
FFN hidden	r	d_{ff}
width		

주의 행/열 convention을 섞으면 permutation-equivariance 식이 $T(XP) = T(X)P$ 인지 $T(PX) = PT(X)$ 인지부터 바뀐다. 둘은 같은 내용이지만 같은 식은 아니다. 이번 노트에서는 원 논문과 동일하게 $token = column$ 을 쓴다.

1.2 이번 주의 세 층위

세 가지 질문을 분리해야 한다.

1. 한 attention layer의 표현력: 주어진 입력 X 에 대해 어떤 attention matrix를 만들 수 있는가?
2. 깊은 Transformer의 함수 표현력: 여러 attention/FFN block을 쌓으면 어떤 seq2seq 함수를 근사할 수 있는가?
3. 학습 가능성: 그 함수를 표현하는 parameter를 SGD가 실제로 찾는가?

이번 주의 universal approximation theorem은 2번에 대한 결과다. 3번을 보장하지 않는다.

핵심 **Expressivity**는 “좋은 parameter가 존재하는가?”의 문제다. **Optimization**은 “그 parameter를 학습 과정이 찾는가?”의 문제다. **Generalization**은 “찾은 함수가 보지 않은 데이터에서도 맞는가?”의 문제다. 이 셋은 서로 다른 theorem이 필요하다.

1.3 이번 주의 결론을 먼저 한 줄로

Yun et al.의 핵심은 다음 두 정리다.

No positional encoding $\implies$ continuous permutation-equivariant functions의 UAT,

Trainable positional encoding $\implies$ arbitrary continuous seq2seq UAT
(on a compact domain).

두 경우 모두 놀라운 점은 attention head 수, head size, FFN width를 크게 만들 필요가 없다는 것이다. 원 논문의 constructive theorem은

$$(h, m, r) = (2, 1, 4)$$

라는 고정 폭으로도 충분하다고 말한다. 대신 필요한 **depth**는 매우 커질 수 있다.

발표에서 이렇게 말하기 “이번 주에는 Transformer를 계산 복잡도나 Turing machine 관점에서 보지 않습니다. 먼저 고정 길이의 연속 함수 근사 문제로 보고, attention이 문맥을 구별하는 식별자를 만들 수 있기 때문에 좁은 폭의 Transformer도 충분히 깊으면 universal하다는 결과를 봅니다. 다만 이는 존재 정리이지 학습 가능성 정리는 아닙니다.”

2

Transformer block의 수학과 permutation symmetry

2.1 원 논문 형태의 self-attention

입력은

$$X = [x_1, \dots, x_n] \in \mathbb{R}^{d \times n}$$

이다. h 개의 attention head를 쓰고 각 head의 크기를 m 이라 하자. i 번째 head에 대해

$$W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{m \times d}, \quad W_O^i \in \mathbb{R}^{d \times m}.$$

Yun et al.은 residual을 포함한 attention layer를

$$\text{Attn}(X) = X + \sum_{i=1}^h W_O^i W_V^i X \text{ softmax}\left((W_K^i X)^\top W_Q^i X\right)$$

로 쓴다. softmax는 각 column에 적용한다.

현대 구현에서 혼한 $1/\sqrt{m}$ scaling은 이 표현력 정리의 본질이 아니다. 고정된 양의 scale은 projection matrix에 흡수할 수 있기 때문이다.

2.2 token-wise FFN

attention 결과를 $Z = \text{Attn}(X)$ 라 쓰면 FFN은

$$\text{FF}(Z) = Z + W_2 \text{ReLU}(W_1 Z + b_1 \mathbf{1}_n^\top) + b_2 \mathbf{1}_n^\top,$$

여기서

$$W_1 \in \mathbb{R}^{r \times d}, \quad W_2 \in \mathbb{R}^{d \times r}.$$

중요한 점은 모든 column에 같은 W_1, W_2 가 적용된다는 것이다. 즉 FFN 자체는 token 끼리 정보를 섞지 않는다.

2.3 왜 positional encoding이 없으면 permutation equivariant 인가

$P \in \mathbb{R}^{n \times n}$ 를 permutation matrix라 하자. 입력 token 순서를 바꾸면 $X \mapsto XP$ 이다.

먼저 query와 key는

$$W_Q(XP) = (W_Q X)P, \quad W_K(XP) = (W_K X)P.$$

따라서 score matrix는

$$\begin{aligned} S(XP) &= (W_K XP)^\top (W_Q XP) \\ &= P^\top (W_K X)^\top (W_Q X)P \\ &= P^\top S(X)P. \end{aligned}$$

column-wise softmax는 같은 permutation에 대해

$$\text{softmax}(P^\top SP) = P^\top \text{softmax}(S)P$$

를 만족한다. 따라서 한 head의 value aggregation은

$$\begin{aligned} W_V(XP) \text{softmax}(S(XP)) &= W_V X P P^\top \text{softmax}(S(X))P \\ &= W_V X \text{softmax}(S(X))P. \end{aligned}$$

결국 residual까지 포함하면

$$\text{Attn}(XP) = \text{Attn}(X)P.$$

FFN도 같은 parameter를 column-wise로 적용하므로

$$\text{FF}(ZP) = \text{FF}(Z)P.$$

따라서 block의 합성 역시

$$T(XP) = T(X)P.$$

명제 2.1 (Permutation equivariance). Positional encoding이 없는 Transformer block과 그 finite composition은 permutation equivariant하다. 즉 모든 permutation matrix P 에 대해

$$T(XP) = T(X)P.$$

보충 유도 — 논문 밖의 독자용 전개 위 유도는 원 논문의 Claim 1을 발표용으로 한 줄씩 풀어쓴 것이다. 핵심은 score가 $P^\top SP$ 로 conjugation되고, value 쪽의 P 와 softmax 쪽의 $P^\top$ 가 곱해져 사라진다는 것이다.

2.4 이 symmetry가 의미하는 제한

예를 들어 positional information이 전혀 없는 상태에서 두 token을 swap하면 출력도 똑같이 swap되어야 한다. 따라서 “첫 번째 위치만 특별히 0으로 보내라” 같은 함수는 표현할 수 없다.

즉 UAT를 말하려면 target function class도 이 symmetry를 가져야 한다. 이것이 Yun et al.의 Theorem 2에서 target을 permutation-equivariant 함수로 제한하는 이유다.

발표에서 이렇게 말하기 “Positional encoding이 없다는 것은 단순히 위치 정보를 조금 덜 쓴다는 뜻이 아닙니다. Architecture 자체가 permutation equivariance라는 정확한 symmetry를 갖게 됩니다. 그래서 UAT의 target class도 먼저 이 symmetry 안으로 제한해야 합니다.”

3

Universal Approximation을 정확히 읽기

3.1 고전적 UAT와 이번 정리의 차이

고전적인 universal approximation theorem은 대략

$$f : K \subset \mathbb{R}^D \rightarrow \mathbb{R}^M$$

인 연속 함수를 neural network가 임의의 오차까지 근사할 수 있다는 존재 정리다. Transformer에서는 입력과 출력이 행렬

$$X \in \mathbb{R}^{d \times n}$$

이므로 전체 공간을 펼치면 $D = dn$ 이다. 하지만 Transformer는 token 간 parameter sharing과 attention 구조를 갖기 때문에 일반 MLP의 UAT를 그대로 적용하는 것으로는 architecture의 역할을 설명하지 못한다.

3.2 L^p 함수 거리

Yun et al.은 두 함수 $f_1, f_2 : \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^{d \times n}$ 사이의 거리를

$$d_p(f_1, f_2) = \left(\int \|f_1(X) - f_2(X)\|_p^p dX \right)^{1/p}$$

로 둔다. 여기서 matrix norm은 entry-wise ℓ_p norm이다.

따라서 “universal”은

$$\forall \varepsilon > 0, \quad \exists \theta \quad \text{s.t.} \quad d_p(f, T_\theta) < \varepsilon$$

라는 뜻이다.

주의 이 결과는 uniform norm $\sup_X \|f(X) - T(X)\|$ 에 대한 theorem이 아니다. 또한 approximation error가 작은 parameter가 존재한다고 말할 뿐, 그 parameter를 polynomial time에 찾을 수 있다는 뜻도 아니다.

3.3 함수 class 정의

$\mathcal{F}_{PE}$ 를 compact support를 갖는 연속 permutation-equivariant 함수들의 class라 하자. 즉

$$f(XP) = f(X)P$$

이고, 충분히 큰 compact set 밖에서는 $f(X) = 0$ 이다.

한편 positional encoding을 넣은 경우에는 compact domain $K \subset \mathbb{R}^{d \times n}$ 위의 임의의 연속 함수 class를 $\mathcal{F}_{CD}$ 라 쓴다.

3.4 Theorem 2: positional encoding 없이도 fixed-width UAT

정리 3.1 (Yun et al., 2020, Theorem 2). $1 \leq p < \infty$ 이고 $\varepsilon > 0$ 라 하자. 모든 $f \in \mathcal{F}_{PE}$ 에 대해

$$g \in \mathcal{T}_{2,1,4}$$

가 존재하여

$$d_p(f, g) \leq \varepsilon$$

를 만족한다.

$\mathcal{T}_{h,m,r}$ 는 head 수 h , 각 head 크기 m , FFN hidden width r 인 Transformer block들을 임의의 깊이로 합성한 함수 class다. 그러므로 정리는

$$h = 2, \quad m = 1, \quad r = 4$$

라는 고정 폭만으로 충분하다고 말한다.

핵심 여기서 “fixed width”는 **fixed depth**가 아니다. h, m, r 는 n, d 와 무관한 상수지만, 원하는 오차가 작아질수록 construction의 depth는 커진다.

3.5 왜 이 결과가 처음 보면 이상한가

두 가지 제약이 동시에 있다.

- FFN은 각 token에 똑같은 함수만 적용한다.
- attention은 pairwise dot product와 shared projection만 이용한다.

그런데도 모든 연속 equivariant seq2seq function을 근사할 수 있다. 이 역설을 푸는 핵심이 contextual mapping이다.

발표에서 이렇게 말하기 “Universal approximation에서 놀라운 숫자는 2, 1, 4입니다. 두 개의 scalar attention head와 폭 4의 FFN이면 충분합니다. 그러나 width가 작다는 말과 network가 작다는 말은 다릅니다. 이 construction은 depth를 크게 써서 표현력을 얻습니다.”

4

증명의 중심: contextual mapping

4.1 왜 token-wise FFN 만으로는 부족한가

두 문장을 생각하자.

$$L^{(1)} = [v_I, v_{\text{am}}, v_{\text{happy}}], \quad L^{(2)} = [v_I, v_{\text{am}}, v_{\text{Bob}}].$$

첫 token “I”의 raw embedding은 두 문장에서 같다. token-wise FFN만 쓰면 같은 입력은 같은 출력으로 보내기 때문에 두 문맥의 “I”를 다르게 처리할 수 없다.

따라서 먼저 전체 sequence를 보고

(sequence, position/token)

조합마다 서로 다른 scalar 또는 vector code를 만들어야 한다.

4.2 Contextual mapping의 정의

유한 집합 $\mathcal{L} \subset \mathbb{R}^{d \times n}$ 에 대해

$$q: \mathcal{L} \rightarrow \mathbb{R}^{1 \times n}$$

가 contextual mapping이라는 것은 다음을 뜻한다.

1. 같은 sequence L 안의 n 개 값 $q(L)_1, \dots, q(L)_n$ 이 모두 다르다.
2. 서로 다른 context L, L' 가 permutation 관계가 아니라면, $q(L)$ 과 $q(L')$ 에 등장하는 값들도 서로 겹치지 않는다.

즉 각 token에 붙은 scalar가

“나는 어느 sequence의 어느 token인가”

를 식별하는 ID가 된다.

핵심 Self-attention의 역할은 최종 함수를 직접 계산하는 것이 아니라, 문맥에 따라 달라지는 고유 식별자를 만드는 것이다. 그 뒤의 token-wise FFN은 이 identifier를 lookup key처럼 사용하여 원하는 출력값을 생성할 수 있다.

4.3 UAT proof의 세 단계

Yun et al.의 Theorem 2 proof는 크게 세 단계다.

Step 1: 연속 함수를 grid 위의 piecewise-constant 함수로 근사

compact support를 편의상 $[0, 1]^{d \times n}$ 안에 둔다. grid size $\delta > 0$ 에 대해

$$\mathcal{G}_\delta = \{0, \delta, 2\delta, \dots, 1 - \delta\}^{d \times n}$$

를 잡는다. 각 grid point L 에 대해 cube

$$S_L = \prod_{j=1}^d \prod_{k=1}^n [L_{jk}, L_{jk} + \delta)$$

를 정의한다.

연속성 때문에 δ 가 충분히 작으면 같은 cell 안에서 함수값의 변화가 작다. 따라서 적절한 상수 행렬 A_L 을 골라

$$\bar{f}(X) = \sum_{L \in \mathcal{G}_\delta} A_L \mathbf{1}\{X \in S_L\}$$

로 근사할 수 있고,

$$d_p(f, \bar{f}) < \varepsilon/3$$

가 되도록 할 수 있다.

보충 유도 — 논문 밖의 독자용 전개 연속 함수의 grid approximation은 Transformer 고유의 부분이 아니다. compact set 위에서 uniform continuity를 사용하면 cell diameter가 충분히 작을 때 함수값 차이가 작아진다. 이번 proof에서 진짜 Transformer-specific한 부분은 Step 2다.

Step 2: modified Transformer로 grid function을 구현

분석을 위해 softmax를 hardmax로, ReLU를 특정 piecewise-linear activation으로 바꾼 modified Transformer를 생각한다. 그 안에서는 다시 세 작업을 수행한다.

1. token-wise FFN으로 입력을 grid point로 **quantize** 한다.
2. self-attention stack으로 각 grid sequence에 대한 **contextual mapping**을 만든다.
3. token-wise FFN으로 contextual ID를 원하는 A_L 의 각 column으로 **value map**한다.

두 번째 단계가 핵심이다.

Step 3: hardmax/piecewise activation을 실제 softmax/ReLU Transformer로 근사

hardmax는 temperature를 높인 softmax로 근사할 수 있다. 예를 들어 최대값이 유일할 때

$$\text{softmax}(\lambda z) \xrightarrow{\lambda \rightarrow \infty} e_{\arg \max z}.$$

또한 필요한 piecewise-linear scalar map은 작은 ReLU subnetwork로 근사/구현할 수 있다. 이 때문에 modified network를 실제 Transformer $\mathcal{T}_{2,1,4}$ 로 옮길 수 있다.

마지막에는 triangle inequality를 사용한다.

$$\begin{aligned} d_p(f, g) &\leq d_p(f, \bar{f}) + d_p(\bar{f}, \bar{g}) + d_p(\bar{g}, g) \\ &\leq \frac{2\varepsilon}{3} + O(\delta^{d/p}). \end{aligned}$$

δ 를 충분히 작게 잡으면 최종 오차가 ε 보다 작아진다.

4.4 Attention이 contextual ID를 만든다는 의미

원 논문의 Lemma 6은 grid의 거의 모든 입력에 대해 $h = 2, m = 1$ attention layer들을 여러 개 쌓아 contextual mapping을 구성한다. 핵심 idea는 한 번의 attention으로 모든 문맥을 완전히

encode한다기보다, 여러 층의 **selective shift**를 통해 grid point의 정보를 순차적으로 scalar code에 누적하는 것이다.

한 column의 scalar label을

$$l_j = u^\top L_{:,j}$$

로 두고, attention operation으로 특정 label만 골라 현재 global max/min 차이에 비례한 shift를 준다. 반복하면 각 token label이 전체 sequence에 의존하는 서로 다른 값

$$\tilde{l}_1 < \tilde{l}_2 < \dots < \tilde{l}_n$$

로 바뀐다. 더 중요한 점은 서로 permutation 관계가 아닌 두 sequence의 label set도 서로 분리되도록 construction할 수 있다는 것이다.

주의 여기서 “attention이 한 층만으로 arbitrary context를 완벽히 encode한다”고 말하면 틀린다. Yun et al.의 UAT construction은 **attention layer**의 깊은 합성을 사용한다.

발표에서 이렇게 말하기 “증명은 세 단계로 보면 됩니다. 먼저 연속 함수를 grid lookup table로 바꾸고, attention이 각 context-token 쌍에 unique ID를 붙이고, FFN이 그 ID를 원하는 출력으로 lookup합니다. 마지막으로 proof 편의를 위해 쓴 hardmax를 softmax로 다시 근사합니다.”

5

Positional encoding이 symmetry를 어떻게 깨는가

5.1 왜 위치 정보가 필요할까

positional encoding이 없으면 항상

$$T(XP) = T(X)P$$

여야 한다. 따라서 arbitrary seq2seq function의 UAT는 불가능하다. 예를 들어

$$f(X)_{:,1} = 0, \quad f(X)_{:,2} = X_{:,2}$$

처럼 특정 absolute position을 특별 취급하는 함수는 permutation equivariant하지 않다.

5.2 trainable positional encoding

Yun et al.은

$$E \in \mathbb{R}^{d \times n}$$

를 trainable positional encoding으로 두고

$$g_P(X) = g(X + E), \quad g \in \mathcal{T}_{h,m,r}$$

인 함수 class를 생각한다.

이제 column j 에는 입력 token뿐 아니라 $E_{:,j}$ 라는 위치별 offset이 더해진다. 따라서 token을 permutation해도 위치 offset이 같이 permutation되는 것이 아니므로 원래의 symmetry가 깨진다.

5.3 Theorem 3

정리 5.1 (Yun et al., 2020, Theorem 3). $1 \leq p < \infty$ 와 $\varepsilon > 0$ 를 고정하자. compact domain 위의 임의의 연속 함수

$$f : K \subset \mathbb{R}^{d \times n} \rightarrow \mathbb{R}^{d \times n}$$

에 대해 trainable positional encoding을 갖는

$$g \in \mathcal{T}_{2,1,4}^P$$

가 존재하여

$$d_p(f, g) \leq \varepsilon$$

를 만족한다.

즉 positional encoding을 통해 permutation-equivariance라는 structural restriction을 제거하면, 고정 길이 n 의 sequence space에서는 arbitrary continuous seq2seq function을 근사할 수 있다.

5.4 “positional encoding이 있으면 universal”의 정확한 범위

다음 세 제한을 반드시 기억해야 한다.

1. **고정된 sequence length n** : theorem은 variable-length generalization theorem이 아니다.
2. **compact domain**: 임의의 무한 domain에서 uniform하게 모든 continuous function을 근사한다는 뜻이 아니다.
3. **trainable PE**: 원 theorem의 construction은 trainable positional encoding을 사용한다.

따라서 이 theorem에서 바로

“sinusoidal PE를 쓰는 실제 LLM도 모든 길이에 대해 universal”

을 결론내릴 수 없다.

주의 UAT와 length generalization을 연결하지 말 것. 고정 n 의 함수 공간에서의 universality와, 훈련보다 긴 sequence에서 algorithm을 계속 수행하는 능력은 다른 문제다. 후자는 뒤 주치의 computational theory / length-generalization 문제에 가깝다.

5.5 PE는 단순한 부가 정보가 아니라 symmetry-breaking device

수학적으로 positional encoding의 중요한 역할은

$$\text{equivariant function class} \rightarrow \text{general function class}$$

로 hypothesis class의 symmetry를 바꾸는 것이다.

이 관점은 image model의 translation equivariance, graph network의 permutation invariance에서도 반복된다. Architecture가 어떤 symmetry를 강제로 보존하는지, positional/node/coordinate feature가 그 symmetry를 어디까지 깨는지를 따지는 것이 표현력 분석의 기본 패턴이다.

발표에서 이렇게 말하기 “Positional encoding의 이론적 역할은 위치를 알려주는 것보다 더 정확히 말하면 symmetry를 깨는 것입니다. PE가 없으면 모델 class 전체가 permutation equivariant라 arbitrary function을 표현할 수 없고, trainable PE를 더하면 그 제약이 사라져 fixed-length compact-domain UAT가 됩니다.”

6

Attention matrix 자체의 표현력과 low-rank bottleneck

6.1 질문을 한 layer 수준으로 내리기

이제 깊은 Transformer 전체가 아니라 한 attention head를 본다. $X \in \mathbb{R}^{d \times n}$ 에 대해 query/key dimension을 d_k 라 하면

$$Q = W_Q X \in \mathbb{R}^{d_k \times n}, \quad K = W_K X \in \mathbb{R}^{d_k \times n}.$$

attention probability matrix를

$$P = \text{softmax}\left(\frac{K^\top Q}{\sqrt{d_k}}\right) \in \mathbb{R}^{n \times n}$$

라 하자.

질문은 다음과 같다.

고정된 입력 X 와 원하는 positive column-stochastic matrix P 가 주어졌을 때, 적절한 W_Q, W_K 를 골라 정확히 그 P 를 만들 수 있는가?

6.2 softmax 역문제

P 의 모든 원소가 양수이고 각 column 합이 1이라 하자. 만약 score matrix S 를

$$S_{ij} = \log P_{ij}$$

로 두면 각 column에 대해

$$\text{softmax}(S)_{ij} = \frac{e^{\log P_{ij}}}{\sum_{r=1}^n e^{\log P_{rj}}} = \frac{P_{ij}}{1} = P_{ij}.$$

따라서 문제는 결국

$$K^\top Q = \sqrt{d_k} S$$

를 만들 수 있는지의 rank/factorization 문제로 바뀐다.

6.3 Representation Theorem

정리 6.1 (Bhojanapalli et al., 2020, Theorem 1의 핵심). $X \in \mathbb{R}^{d \times n}$ 이 full column rank이고 $d \geq n$ 이라 하자. query/key projection dimension이 충분히 커서 원하는 $n \times n$ score matrix를 factorize할 수 있으면, 임의의 positive column-stochastic matrix P 에 대해 적절한 W_Q, W_K 가 존재하여

$$\text{softmax}\left(\frac{(W_K X)^\top (W_Q X)}{\sqrt{d_k}}\right) = P$$

를 만족한다. 반대로 projection dimension이 sequence length보다 작으면 일반적으로 표현할 수 없는 (X, P) 가 존재한다.

6.4 충분한 차원에서의 explicit construction

X 가 full column rank 이므로 left inverse

$$X^\dagger = (X^\top X)^{-1} X^\top$$

가 존재하고

$$X^\dagger X = I_n.$$

원하는 score matrix를 S 라 하자. S 를

$$S = \tilde{K}^\top \tilde{Q}$$

로 factorize 할 수 있다고 하자. 그 다음

$$W_K = \tilde{K} X^\dagger, \quad W_Q = \tilde{Q} X^\dagger$$

로 두면

$$\begin{aligned} (W_K X)^\top (W_Q X) &= (\tilde{K} X^\dagger X)^\top (\tilde{Q} X^\dagger X) \\ &= \tilde{K}^\top \tilde{Q} \\ &= S. \end{aligned}$$

따라서 softmax를 통해 원하는 P 를 얻는다.

보충 유도 — 논문 밖의 독자용 전개 원 논문은 softmax의 column-wise additive constant 불변성까지 포함해 construction을 쓴다. 발표에서는 “left inverse로 입력 X 의 영향을 제거한 뒤 원하는 log-attention matrix를 query/key inner product로 factorize 한다”고 설명하면 구조가 가장 잘 보인다.

6.5 왜 $d_k < n$ 이면 bottleneck이 생기는가

score matrix는

$$S = K^\top Q$$

이므로

$$\text{rank}(S) \leq d_k.$$

$d_k < n$ 이면 모든 $n \times n$ matrix를 만들 수 없다. softmax가 nonlinear 이긴 하지만, 그 전 단계 logit 이 low-rank factorization을 거쳐야 한다는 structural constraint는 남는다. 원 논문은 실제로 특정 (X, P) 가 어떤 W_Q, W_K 로도 표현되지 않음을 보인다.

6.6 multi-head에서 생기는 직관

표준 heuristic에서는 embedding dimension d_{model} 을 head 수 h 로 나누어

$$d_k \approx \frac{d_{\text{model}}}{h}$$

로 둔다. h 를 늘리면 각 head의 dimension이 작아진다. 따라서 head 수를 무작정 늘리는 것이 개별 head의 표현력을 항상 키우는 것은 아니다.

주의 “ $d_k < n$ 이면 Transformer가 universal하지 않다”고 결론내리면 안 된다. Bhojanapalli et al.의 theorem은 한 head가 한 입력에서 **arbitrary attention matrix**를 정확히 표현하는 능력에 관한 결과다. Yun et al.의 UAT는 오히려 $m = 1$ 인 작은 head를 깊게 여러 층 쌓아 universality를 얻는다. 두 결과는 모순이 아니라 분석 대상이 다르다.

발표에서 이렇게 말하기 “한 layer만 보면 head dimension이 작을 때 rank bottleneck이 분명히 있습니다. 그런데 전체 network의 universality와 한 layer의 arbitrary attention-matrix expressivity는 다른 개념입니다. Yun의 결과는 작은 head를 깊게 조합해 이 한 층의 제한을 우회합니다.”

7

Depth와 Width: fixed-width universal의 실제 비용

7.1 폭이 작다는 것과 효율적이라는 것은 다르다

Theorem 2의 핵심 숫자는 $(h, m, r) = (2, 1, 4)$ 다. 그러나 approximation tolerance를 줄일수록 grid resolution δ 를 작게 해야 하고, 필요한 block 수는 빠르게 증가한다.

원 논문의 construction은 대략

$$O\left(n \frac{(1/\delta)^{dn}}{n!}\right)$$

개의 Transformer block을 요구한다. positional encoding을 넣은 general continuous function construction도

$$O\left(n(1/\delta)^{dn}\right)$$

수준의 깊이를 사용할 수 있다.

따라서 이 theorem은

constant width suffices

를 보여주지만

polynomial-size efficient representation

을 보여주는 theorem은 아니다.

7.2 왜 curse of dimensionality가 보이는가

input space의 dimension은 펼치면 dn 이다. 한 축마다 grid spacing이 δ 이면 cell 수가

$$(1/\delta)^{dn}$$

으로 증가한다. arbitrary continuous function을 lookup-table식으로 근사하면 이 exponential dependence는 자연스럽다.

즉 universal approximation은 매우 약한 의미의 “가능성”이다. 실제 언어 함수가 arbitrary worst-case continuous function보다 훨씬 structure가 많기 때문에 실제 모델이 훨씬 적은 parameter로 작동할 수 있는 것이다.

7.3 Depth의 역할

Yun proof에서 depth는 단순한 반복이 아니라 세 가지 계산을 순차적으로 수행하는 resource다.

1. 여러 FFN layer가 coordinate별 quantization을 수행한다.
2. 여러 attention layer가 selective shift를 반복해 context ID를 만든다.
3. 여러 FFN layer가 많은 ID-to-output mapping을 구현한다.

따라서 “attention 한 번이 모든 관계를 parallel하게 본다”는 실무적 직관과 “UAT construction 이 깊이를 많이 필요로 한다”는 수학적 사실은 동시에 참이다.

7.4 width condition을 어떻게 읽을 것인가

두 paper의 width 관련 문장을 함께 놓으면 다음처럼 구분된다.

질문	필요한 width 조건	해석
깊은 Transformer 전체의 UAT	$h = 2, m = 1, r = 4$ 로 충분	depth가 arbitrary하게 커질 수 있음
한 attention head가 arbitrary P 표현	head dimension이 sequence length 수준이면 충분	single-layer exact attention expressivity
실제 효율적 모델 크기	위 theorem들만으로 결정 불가	data/task structure와 optimization이 필요

핵심 표현력에서 width와 depth는 대체 가능한 resource일 수 있다. “small width is universal”은 “width는 중요하지 않다”가 아니라 “충분한 depth가 주어진다면 width를 상수로 고정해도 존재 정리는 성립한다”는 뜻이다.

발표에서 이렇게 말하기 “Fixed-width UAT의 가장 중요한 caveat는 depth입니다. 폭 4의 FFN과 scalar head만으로 충분하지만, proof는 grid cell 수에 가까운 엄청난 depth를 사용할 수 있습니다. 따라서 universality와 parameter efficiency를 같은 말로 쓰면 안 됩니다.”

8

UAT가 말하지 않는 것

8.1 1. SGD learnability를 보장하지 않는다

정리는

$$\exists \theta : d_p(f, T_\theta) < \varepsilon$$

를 말한다. 하지만 실제 training은

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L(\theta_t)$$

로 움직인다. 존재하는 좋은 θ 가 optimization landscape에서 접근 가능한지는 별도 문제다.

지난 주 NTK 결과는 optimization dynamics의 한 regime을 다루었지만, 이번 UAT와 논리적으로 독립이다.

8.2 2. sample complexity를 보장하지 않는다

function class가 universal하면 오히려 너무 많은 함수를 표현할 수 있다. 유한한 data만 보고 target을 식별하려면 capacity control, norm, margin, prior, algorithmic bias 같은 추가 구조가 필요하다.

8.3 3. length generalization을 보장하지 않는다

Theorem 3는 고정된 n 에 대한 함수 공간의 결과다. 훈련 시 길이 $n \leq N$ 만 보고 $n \gg N$ 에서도 같은 알고리즘을 수행할 수 있는지는 전혀 다른 문제다.

8.4 4. computational complexity를 보장하지 않는다

어떤 함수를 표현할 수 있다는 것과 그 함수를 깊이/precision/시간 제한 아래 계산할 수 있다는 것은 다르다. Turing completeness, circuit class TC^0 , formal-language recognition 같은 결과는 다음 computational theory 주제에서 다루는 것이 맞다.

8.5 5. attention weight의 해석 가능성을 보장하지 않는다

attention matrix를 다양하게 표현할 수 있다는 결과는 attention weight가 유일한 explanation이라는 뜻이 아니다. value projection과 residual, 여러 head/layer의 조합 때문에 최종 output과 attention weight의 관계는 별도 분석이 필요하다.

주의 발표에서 가장 위험한 문장은 “Transformer는 universal하므로 어떤 문제든 풀 수 있다”이다. 정확한 문장은 “고정 길이의 compact continuous seq2seq function class에 대해 충분한 depth와 적절한 positional encoding을 허용하면 arbitrary accuracy의 approximation parameter가 존재한다”이다.

8.6 Expressivity, computability, learnability, generalization

개념	질문	대표 수학적 형태
Expressivity	표현 가능한가?	$\exists \theta : \ T_\theta - f\ < \varepsilon$
Computability	제한된 resource로 계산 가능한가?	depth/precision/circuit/language class
Learnability	data와 optimization으로 찾을 수 있는가?	sample/optimization complexity
Generalization	unseen input에서도 맞는가?	population risk, OOD/length behavior

발표에서 이렇게 말하기 “이번 주 theorem은 architecture의 가능 영역을 정해줍니다. 하지만 가능 영역 안에서 SGD가 무엇을 찾는지, 데이터 몇 개가 필요한지, 더 긴 sequence에서도 되는지는 하나도 자동으로 따라오지 않습니다. 그 분리가 이후 이론 세미나의 출발점입니다.”

9

발표 준비용 압축 정리

9.1 칠판에 반드시 남길 여덟 식

1. Transformer input convention

$$X \in \mathbb{R}^{d \times n}.$$

2. Self-attention score

$$S = (W_K X)^\top (W_Q X).$$

3. Permutation equivariance

$$T(XP) = T(X)P.$$

4. UAT without PE

$$\forall f \in \mathcal{F}_{\text{PE}}, \forall \varepsilon > 0, \quad \exists g \in \mathcal{T}_{2,1,4} : d_p(f, g) \leq \varepsilon.$$

5. Positional encoding

$$g_P(X) = g(X + E).$$

6. UAT with trainable PE

$$\forall f \in \mathcal{F}_{\text{CD}}, \forall \varepsilon > 0, \quad \exists g \in \mathcal{T}_{2,1,4}^P : d_p(f, g) \leq \varepsilon.$$

7. Attention-logit rank bound

$$\text{rank}(K^\top Q) \leq d_k.$$

8. Fixed width does not mean fixed size

$$\text{width} = O(1), \quad \text{depth can grow as } (1/\delta)^{dn}.$$

9.2 발표 순서 추천

1. Transformer block 수식을 한 번만 정확히 정의한다.
2. PE가 없을 때 permutation equivariance를 직접 4줄로 증명한다.
3. UAT의 target class와 d_p 를 정의한다.
4. Yun Theorem 2의 (2, 1, 4) result를 먼저 보여준다.
5. proof를 quantization $\rightarrow$ contextual mapping $\rightarrow$ value mapping의 세 단계로 설명한다.
6. trainable PE가 symmetry를 깨고 Theorem 3으로 이어짐을 보여준다.
7. Bhojanapalli의 rank theorem으로 한 layer의 attention expressivity를 보완한다.
8. 마지막에 fixed-width universality와 huge depth, learnability/length generalization의 차이를 분명히 한다.

9.3 자주 나오는 질문

Q1. PE가 없으면 Transformer가 순서를 전혀 못 보나?

absolute position을 구분하지 못하고 permutation equivariant하다. 다만 token content 자체에 순서 정보가 이미 encode되어 있다면 그 정보는 사용할 수 있다. 순수 architecture symmetry에 대한 진술이다.

Q2. 두 head, head size 1이면 실제 Transformer도 충분하다는 뜻인가?

표현 가능성의 존재 정리에서는 충분하다. 그러나 필요한 depth가 매우 클 수 있어 practical efficiency를 의미하지 않는다.

Q3. contextual mapping은 그냥 positional encoding인가?

아니다. contextual mapping은 같은 token도 전체 sequence가 달라지면 다른 ID를 갖게 하는 *content-dependent* mapping이다. PE는 position-dependent offset으로 symmetry를 깨는 장치다.

Q4. 한 head에서 $d_k < n$ 이면 왜 UAT와 모순이 아닌가?

single-head single-layer가 arbitrary attention matrix를 정확히 만드는 능력과, 깊은 network가 arbitrary function을 근사하는 능력은 다른 property다. 깊은 합성은 한 층의 rank restriction을 우회할 수 있다.

Q5. UAT와 Turing completeness는 같은 말인가?

아니다. UAT는 연속 공간의 함수 근사 문제이고, Turing completeness는 discrete computation과 unbounded computation model의 문제다. 서로 하나가 다른 하나를 자동으로 함의하지 않는다.

Q6. positional encoding이 sinusoidal이어도 Theorem 3가 그대로 성립하나?

원 theorem은 trainable E 를 허용한다. fixed sinusoidal PE에 대해 동일한 statement를 그대로 인용하면 안 된다.

9.4 흔한 수학적 실수 체크리스트

- row-token / column-token convention을 섞지 않았는가?
- PE가 없는 Transformer의 target class를 arbitrary function으로 쓰지 않았는가?
- Theorem 2의 metric이 L^p 함수 거리임을 잊지 않았는가?
- $(h, m, r) = (2, 1, 4)$ 를 fixed depth라고 오해하지 않았는가?
- contextual mapping을 한 attention layer의 결과라고 말하지 않았는가?
- proof의 hardmax network와 실제 softmax Transformer를 구분했는가?
- trainable PE theorem을 sinusoidal/RoPE에 자동 적용하지 않았는가?
- single-layer low-rank bottleneck을 전체 deep Transformer의 non-universality로 확대하지 않았는가?
- UAT를 optimization/generalization theorem처럼 설명하지 않았는가?
- fixed-length UAT를 length generalization claim으로 바꾸지 않았는가?

9.5 30초 결론

Positional encoding이 없는 Transformer는 architecture 자체가 permutation equivariant하므로 그 symmetry를 가진 함수만 표현한다. 그 범위 안에서는 Yun et al.이 두 scalar head와 FFN width 4라는 고정 폭만으로도 arbitrary continuous equivariant seq2seq function을 근사할 수

있음을 보였다. 증명의 핵심은 attention stack이 context-dependent unique identifier를 만들고 token-wise FFN이 이를 원하는 output으로 바꾸는 것이다. Trainable positional encoding을 추가하면 permutation symmetry가 깨져 고정 길이 compact domain의 arbitrary continuous seq2seq function에 대한 UAT로 확장된다. 그러나 fixed-width universality는 매우 큰 depth를 허용하는 존재 정리이며, SGD learnability, sample efficiency, computational complexity, length generalization을 보장하지 않는다.

발표에서 이렇게 말하기 “Transformer UAT의 핵심은 attention이 모든 함수를 직접 계산한다는 것이 아니라 context를 구별할 수 있다는 데 있습니다. Attention이 각 context-token 쌍에 고유한 ID를 만들면, 이후 token-wise FFN은 그 ID를 lookup key처럼 써서 arbitrary output을 만들 수 있습니다. PE는 여기에 absolute position까지 구별할 수 있도록 symmetry를 깨줍니다.”

참고문헌 및 원문에서 확인할 위치

Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, Sanjiv Kumar. “Are Transformers universal approximators of sequence-to-sequence functions?” ICLR 2020.

- Sec. 2: Transformer block 정의와 permutation equivariance Claim 1.
- Theorem 2: $\mathcal{T}_{2,1,4}$ 의 continuous permutation-equivariant UAT.
- Theorem 3: trainable positional encoding을 추가한 arbitrary continuous seq2seq UAT.
- Def. 3.1, Lemma 6: contextual mapping과 self-attention construction.
- Sec. 3.2–4: quantization, contextual mapping, value mapping의 proof skeleton.
- Sec. 4.4: fixed width 대신 매우 큰 depth가 필요한 construction의 size discussion.

<https://arxiv.org/abs/1912.10077>

Srinadh Bhojanapalli, Chulhee Yun, Ankit Singh Rawat, Sashank J. Reddi, Sanjiv Kumar. “Low-Rank Bottleneck in Multi-head Attention Models.” ICML 2020.

- Sec. 2 / Theorem 1: 한 attention head가 arbitrary positive stochastic attention matrix를 표현할 수 있는 dimension 조건.
- Proof of Theorem 1: $X^\dagger X = I$ 를 이용해 input dependence를 제거하고 원하는 score matrix를 구성.
- Low-rank case: head projection dimension이 sequence length보다 작을 때 발생하는 representation restriction.
- Sec. 3–4: standard multi-head heuristic에서 head 수 증가와 head dimension 감소 사이의 tradeoff.

<https://proceedings.mlr.press/v119/bhojanapalli20a.html>

주의— 원문과 이 노트의 경계 Permutation-equivariance의 행렬 유도, softmax inverse를 $\log P$ 로 보는 설명, grid approximation의 uniform-continuity 직관, attention score의 rank bound를 통한 bottleneck 설명은 원 theorem을 이해하기 위해 한 줄씩 풀어쓴 보충 전개다. Yun et al.의 full contextual-mapping proof는 Appendix에서 매우 긴 selective-shift construction을 사용하므로, 이 노트는 그 구조와 theorem statement를 보존하되 모든 index inequality를 복제하지 않았다. 또한 UAT는 고정 길이의 함수 근사 결과이며 Turing completeness나 length generalization 결과로 확장해 해석하지 않는다.